

SmartAutoTech Backend Enforcement Logic

Below is the **backend enforcement layer** that SmartAutoTech needs so that the GPT cannot break the production flow.

SmartAutoTech Backend Enforcement Logic

1. Store diagnostic state outside GPT

Your backend should track:

```
{  
  "session_id": "abc123",  
  "vehicle": {  
    "year": "2015",  
    "make": "Toyota",  
    "model": "Corolla",  
    "engine": "1.6L"  
  },  
  "active_concern": "headlights do not work",  
  "paused_concerns": [],  
  "current_step": 1,  
  "expected_response_options": [  
    "Power present",  
    "No power",  
    "Voltage present but drops under load"  
  ],  
  "last_valid_step": 1,  
  "status": "active"  
}
```

The GPT should **not be trusted** to control session state by itself.

2. Backend response gate

Before sending technician input to GPT, the backend checks:

Is this a new session?

→ require Year / Make / Model / Engine / Concern

Is this an active session?

→ compare technician answer to expected response options

Does answer match?

→ send to GPT and allow next step

Does answer not match?

→ reject or force option selection

Is answer vague?

→ reject and repeat same step

Is answer a new concern?

→ ask switch confirmation before GPT continues

3. Required API fields from GPT

Force GPT to return structured JSON behind the scenes:

```
{  
  "vehicle": "2015 Toyota Corolla 1.6L",  
  "concern": "headlights do not work",  
  "system_focus": "Electrical",  
  "current_assessment": "Headlight failure requires load-side and feed-side separation.",
```

```
"step_number": 1,  
"step_title": "Headlight Connector Power Test",  
"instruction": "With headlights commanded ON, backprobe the low-beam connector power terminal  
using a DVOM.",  
"what_to_check": "Measure voltage at the connector under load.",  
"response_options": [  
  "Battery voltage present",  
  "No voltage present",  
  "Voltage present but drops under load"  
],  
"state_action": "awaiting_response"  
}
```

Then your frontend renders it into the clean technician format.

4. Backend blocks bad technician input

Example technician says:

looks good

Backend response:

```
{  
  "accepted": false,  
  "reason": "Invalid test result",  
  "message": "Visual or vague confirmation is not valid. Select one of the listed response options or  
provide a measured result."  
}
```

Do **not** send that to GPT as a valid diagnostic result.

5. Step number enforcement

Backend rule:

If GPT returns step_number less than or equal to current_step after valid response:

Reject GPT response.

Retry with correction message:

“Continue with Step X. Do not reuse prior step number.”

Example:

```
if (gpt.step_number !== session.current_step + 1 && inputWasValid) {  
  throw new Error("Step number violation");  
}
```

6. New concern enforcement

If technician says:

also AC not working

Backend should not let GPT reset.

Backend returns:

```
{  
  "state_action": "confirm_switch",  
  "message": "Confirm switch to new concern: AC not working? Reply Switch or Continue current concern."  
}
```

If technician replies:

Switch

Backend updates:

```
{  
  "paused_concerns": [  
    {  
      "concern": "headlights do not work",  
      "paused_at_step": 3  
    }  
  ]  
}
```

```
],  
  "active_concern": "AC not working",  
  "current_step": 0  
}
```

Then GPT starts:

Previous concern paused. Starting new diagnostic.

7. Production architecture

Technician Input

↓

Backend Validator

↓

Session State Manager

↓

GPT Prompt Builder

↓

OpenAI API

↓

GPT JSON Output

↓

Backend Output Validator

↓

Frontend Display

The strongest version is:

GPT suggests the next diagnostic move.

Backend enforces structure, step count, options, and session state.

Bottom line

Your v5 prompt is strong, but production reliability comes from this rule:

Never let GPT own the diagnostic state.

Let GPT handle diagnostic reasoning.

Let the backend control:

- vehicle lock
- active concern
- step number
- valid response options
- pause/resume
- switch concern
- final conclusion permission